THAAP

Agent Trust Runtime

Product Strategy, Security Architecture and Implementation Roadmap

North Star

Every autonomous economic action should be attributable, authorized, constrained, explainable and recoverable.

Core thesis

THAAP should own the boundary between "an AI agent wants to act" and "the financial system permits that action." It should be model-neutral, agent-neutral, protocol-neutral and payment-rail-neutral.

Working strategy document | Version 1.2 | October 2026 | Transaction-state and Neo4j consistency revision

THAAP = Threat Hunting & Adaptive Analytics for Payments

Wajo/Fo is used in this document as an illustrative market signal and integration example, not as a partner or dependency.

1. Executive Summary

Agentic commerce changes the core assumption of digital payments: the person who owns the money may no longer be the entity directly clicking "buy."

THAAP should evolve from an AI-native fraud/risk platform into the trust and authorization control plane for autonomous economic actors. Its job is not to make an agent smarter; its job is to determine whether an agent is allowed to act, under what constraints, with what evidence, and what should happen when the action is risky or ambiguous.

- **Wedge:** Agent Authorization Gateway returning ALLOW, DENY, STEP-UP, REPLAN or REQUEST_EVIDENCE for sensitive actions.
- **Primitive: Cryptographically verifiable mandates that express user intent, delegation, scope, limits, approval thresholds and expiry - backed by an executor that enforces those permissions at the point of action.**
- **Moat:** A cross-ecosystem Trust Graph connecting principals, agents, providers, merchants, credentials, tools and transactions.
- **Risk layer:** Agent-behavior analytics, semantic intent matching, prompt-injection/compromise detection and transaction fraud signals.
- **Evidence layer: Decision Ledger + Replay that reconstructs what was authorized, what the agent proposed, any step-up approval, budget reservation, credential issuance, what executed, and how timeouts or uncertain outcomes were reconciled.**
- **Positioning:** Do not build the shopping agent or the settlement rail. Build the independent trust layer between intelligence and money.

Strategic positioning

Intelligence providers (OpenAI, Anthropic, Google, Wajo and others) decide what to do. Payment rails (banks, networks, PSPs, wallets and stablecoin infrastructure) move the money. THAAP decides whether the proposed action is trustworthy enough to be permitted.

Target market architecture

AGENT / INTELLIGENCE	THAAP TRUST CONTROL PLANE	MONEY / EXECUTION RAILS
OpenAI	Identity	Banks
Claude	Mandates	Visa / Mastercard
Gemini	Policy	UPI / wallets
Wajo/Fo	Risk	Stripe / PSPs
Enterprise agents	Evidence	Stablecoins
	Threat hunting	
proposed action ->	**decision / credential ->**	**execution + outcome**
Probabilistic planning	*Deterministic controls + adaptive risk*	*Settlement / fulfillment*

2. Why This Matters Now: The Wajo/Fo Signal

Wajo is useful to THAAP not because THAAP should copy Fo, but because Fo exposes the security and authorization problems that appear when software agents become economic actors.

Fo is an action agent that can call, email, coordinate with people, use one-time payment cards, and bring in human assistance when the AI cannot complete a task. Wajo also describes agents with their own inbox, phone number, voice and credit card. [1][2]

The key THAAP implication: once an agent can communicate, authenticate and spend on behalf of a user, the industry needs a separate trust layer that can verify identity, delegation, intent, constraints, risk and accountability before execution.

Wajo/Fo capability	New trust problem	THAAP opportunity
Agent can pay	Was this exact purchase actually authorized?	Mandate verification + scoped execution credential
Agent has its own identity	How does a merchant/bank know which agent is acting?	Agent Identity Registry + attestation
Agent can call/email	Did the agent expose data or follow malicious instructions?	Behavior monitoring + prompt-injection/compromise detection
Agent can act autonomously	How do we enforce limits when the human is absent?	Open mandate + deterministic policy runtime
Agent can use human fallback	Who performed the final action and under whose authority?	Delegation chain + actor provenance
Agent can use commerce protocols	How do we normalize trust across protocols/providers?	Protocol adapters + common THAAP authorization API

> **Important boundary**
>
> THAAP should not become a Wajo competitor. Wajo-like systems are potential clients, partners or upstream action agents. THAAP should provide the authorization and risk infrastructure they can call before sensitive economic actions.

Industry validation

The broader payment ecosystem is converging on the same primitives: explicit user intent, agent identity, constrained credentials, cryptographic evidence and deterministic verification. Google AP2 defines open/closed mandates and receipts; Mastercard Verifiable Intent links identity, intent and action; Visa Intelligent Commerce describes agent-bound credentials, authentication and controls; Stripe Shared Payment Tokens scope payments by seller, amount and time. [4][5][6][7]

3. Problem Definition

Traditional fraud systems were designed for a world in which the account owner or cardholder directly initiated the transaction. Agentic systems break that assumption.

Question	Traditional answer	Agentic-era requirement
Who acted?	Cardholder / logged-in user	Human principal + agent + provider + runtime identity
What was authorized?	Checkout click / OTP / cardholder authentication	Machine-readable mandate representing delegated intent
What can the actor do?	Account permissions / card limits	Fine-grained policy: merchant, category, purpose, time, amount, recurrence, substitutions
Is it safe?	Transaction risk score	Transaction risk + agent trust + semantic intent match + behavior anomaly
What if it goes wrong?	Dispute and logs	Cryptographic evidence + full decision replay + delegation provenance

The THAAP problem statement: Given a principal, an agent, a mandate, a proposed economic action and current risk context, determine whether to permit execution and produce verifiable evidence for that decision.

4. Product North Star and Design Principles

Principle	Meaning in THAAP
Attributable	Every action has a provable actor chain: principal -> agent -> provider/runtime -> tool/credential.
Authorized	An action must trace to authenticated, explicit user or business delegation.
Constrained	Authorization is least-privilege: purpose, amount, merchant, category, time, recurrence and data-sharing boundaries.
Explainable	Every decision records machine-readable reasons, evidence, policy results and model outputs.
Recoverable	The platform supports step-up, revocation, credential kill-switches, dispute evidence and safe retries.
Deterministic at the boundary	LLMs can interpret and score; deterministic code must enforce money-moving controls.
Protocol-neutral	AP2/UCP/network/PSP-specific mechanisms are adapters, not the core domain model.

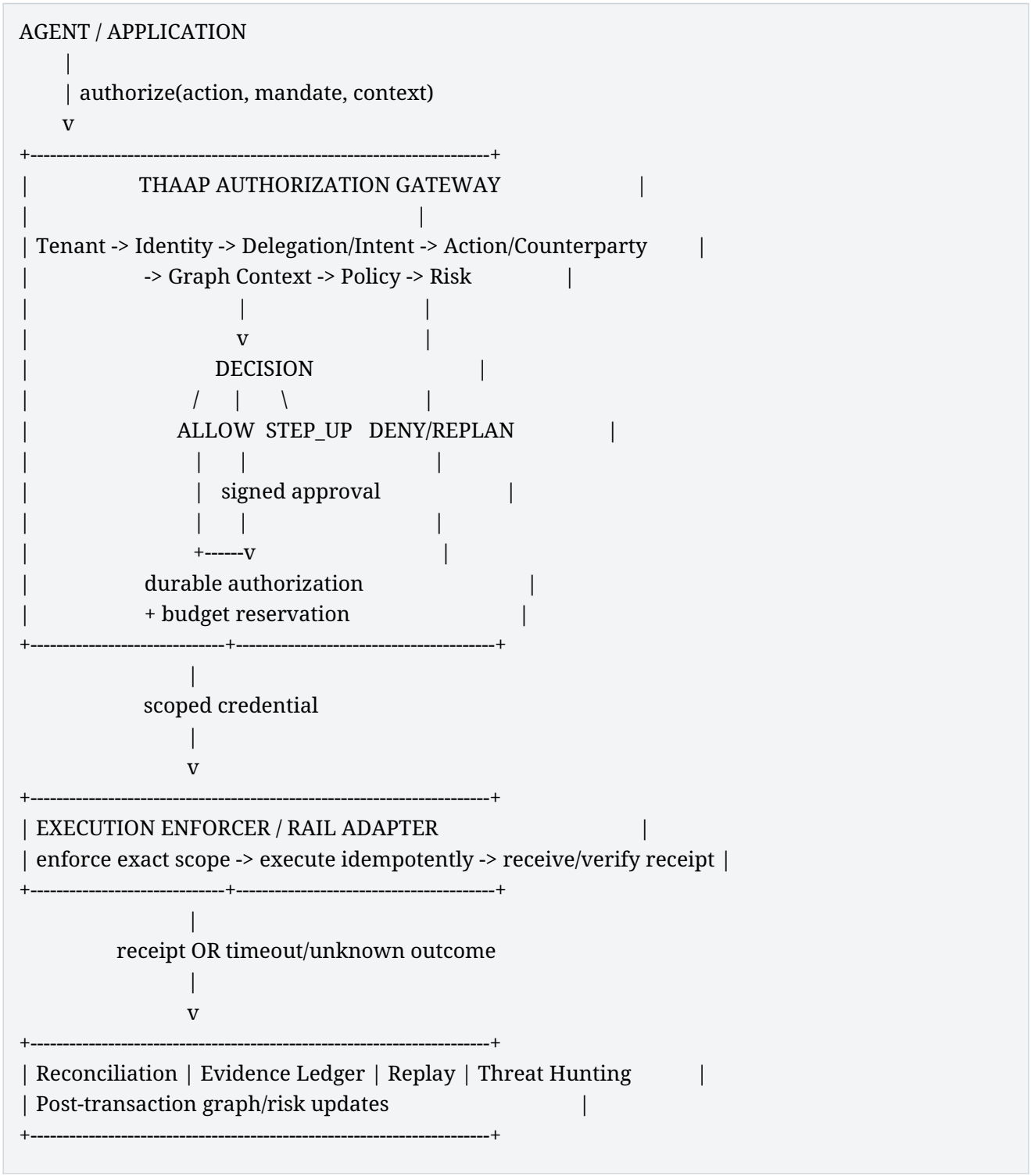
Privacy-minimizing	Reveal only the information required to evaluate the requested action.

Product definition

THAAP Agent Trust Runtime = Identity + Delegation + Intent + Policy + Risk + Evidence + Threat Hunting, exposed through one real-time authorization plane.

5. Reference Architecture

The runtime is split into a synchronous authorization path and asynchronous intelligence/evidence paths. The synchronous path must remain small, deterministic, observable and fail-safe.



5.1 Core services

Service	Primary responsibility	MVP?

Authorization Gateway	Single entry point for agent-sensitive economic actions; establishes tenant context first, then idempotency, caller authentication, schema validation and routing.	YES
Agent Identity Registry	Agent/provider identity, public keys, capabilities, attestations, versions and revocation state.	YES
Principal & Delegation Service	Human/business principal, delegation chain, consent record, revocation.	YES
Mandate / Intent Service	Create, sign, store, validate and close open mandates; semantic constraints.	YES
Policy Runtime	Deterministic evaluation of hard constraints and regulatory/business policy.	YES
Risk Engine	Agent trust score, transaction risk, behavioral anomalies and confidence.	YES, narrow
Step-Up Orchestrator	Executes human/passkey/biometric/bank approval only after policy + graph-informed risk selects STEP_UP; persists signed approval and returns to authorization for re-evaluation.	YES
Decision Ledger	Immutable append-only decision/evidence records with integrity verification.	YES
Decision Replay	Reconstruct policy version, signals, inputs, outputs and receipts.	YES
Trust Graph	Neo4j relationship model and graph-derived features for principal-agent-provider-counterparty-credential-action history. It is queried before adaptive risk/step-up; PostgreSQL remains authoritative for transactional state.	YES - Neo4j in first demo/MVP

Agent SOC / Threat Hunting	Detect compromised agents, prompt injection, tool misuse and abnormal delegation using independent telemetry/provenance. Constraint enforcement is MVP; confirmed prompt-injection detection requires separate evidence.	Phase 2
Protocol Gateway	AP2/UCP/network/PSP adapters; maps external artifacts to THAAP canonical model.	Phase 2
Tenant Isolation Control	Establish tenant_id at ingress; tenant-scoped authorization, policy namespace, storage keys/queries, caches, evidence and graph edges. No cross-tenant reads on the hot path.	YES - day one
Authorization & Budget Reservation	Persist durable authorization state and atomically reserve mandate budget/usage before a usable credential can exist. Release/commit reservations on terminal outcomes.	YES
Execution Enforcer & Reconciliation	Enforce the scoped credential at the execution boundary; make rail calls idempotently; reconcile timeout/unknown outcomes before any retry; bind receipts back to the decision.	YES (real state machine; adapter may be mocked)

5.2 Synchronous decision path

1. **Establish tenant context. Authenticate the caller and bind every subsequent lookup, policy, cache key, evidence record and graph edge to tenant_id.**
2. **Authenticate agent and resolve principal/delegation. Verify provider/runtime identity, request integrity and the active delegation chain.**
3. **Validate mandate and durable authority. Verify signature, expiry, scope, revocation, proof-of-possession, usage/cumulative budget state and constraint schema.**
4. **Normalize instruction and proposed economic action. Convert protocol-specific requests into THAAP canonical intent/action semantics.**
5. **Resolve counterparty and graph context. Load tenant-scoped relationships/history for principal, agent, merchant/payee, credential and prior actions before risk/step-up.**
6. **Run deterministic policy. Enforce hard constraints: merchant, amount, currency, category, purpose, geography, time, recurrence, disclosure and approval thresholds.**

7. **Run adaptive risk. Evaluate transaction risk, behavior anomaly, semantic intent match, reputation, environment and independently sourced compromise signals.**
8. **Resolve preliminary decision. ALLOW, ALLOW_WITH_LIMITS, STEP_UP, REPLAN, REQUEST_EVIDENCE, DENY or QUARANTINE.**
9. **If STEP_UP, execute approval and persist signed approval evidence; then re-enter authorization and re-evaluate relevant policy/risk against the approved action.**
10. Commit durable authorization and atomically reserve budget/usage. A usable credential must not be issued from an ephemeral decision or without a successful reservation.
11. Issue the least-privilege execution credential, bound to authorization_id, reservation_id, exact action/counterparty, proof-of-possession, expiry and usage count.
12. Execute through an enforcing adapter. The executor must reject any use outside the credential scope and use rail-level idempotency keys.
13. Record receipt or enter UNKNOWN/PENDING_RECONCILIATION on timeout. Never blindly retry an uncertain payment; query the rail/merchant until a terminal state is established, then commit or release the reservation.
14. Append evidence and post-transaction updates. Persist decision inputs/hashes, policy/model versions, approval, reservation, credential, execution receipts, reconciliation and graph/risk outcomes.

5.3 Sequencing reconciliation with prior THAAP research

The research baseline remains: Identity -> Intent -> Instruction -> Counterparty -> Authorization -> Transaction -> Post-transaction. The Trust Graph is not conceptually downstream of step-up. Graph context and relationship features are inputs to policy/risk; STEP_UP is a decision outcome produced after that evaluation and executed before final authorization/credential issuance.

Implementation reconciliation: the agreed demonstration and MVP use Neo4j for the Trust Graph from the first implementation. PostgreSQL remains the authoritative store for transactional state such as mandates, approvals, authorizations, budget reservations and executions; Neo4j is the tenant-scoped relationship/feature store used before risk and step-up. THAAP may project graph events through an outbox/Kafka path, but it must not substitute relational edge tables for the agreed Neo4j demonstration.

Canonical runtime order: Tenant -> Identity -> Intent/Delegation -> Instruction/Action -> Counterparty -> Graph Context -> Policy/Risk -> Preliminary Decision -> Step-Up if required -> Re-evaluation -> Durable Authorization + Budget Reservation -> Credential -> Enforced Execution -> Receipt/Reconciliation -> Post-transaction graph update.

6. Canonical Domain Model

The canonical model is the key to staying protocol-neutral. External standards map into these primitives rather than becoming the internal architecture.

Entity	Key fields / semantics
Principal	tenant_id, principal_id, type, authentication strength, jurisdiction, organization/account links
Agent	tenant_id, agent_id, provider_id, public key, version, capabilities, runtime attestation, status

Delegation	tenant_id, principal -> agent/provider authority, scopes, issuance time, expiry, revocation
Mandate	tenant_id, goal, constraints, allowed action types, disclosure policy, proof, open/closed state
EconomicAction	tenant_id, purchase/transfer/subscription/refund/beneficiary/etc.; amount, merchant, asset, destination, purpose
Credential	tenant_id, credential_id, issuer, scope, binding, expiry, usage count, revocation state
RiskContext	tenant_id, behavioral, device/runtime, merchant, network, fraud and threat intelligence signals
Decision	tenant_id, outcome, reasons, policy version, scores, required next action, expiry
Receipt	tenant_id, what executed, when, by whom, external processor refs, hashes/signatures
EvidenceBundle	tenant_id, mandate + action + decision + receipts + provenance used for replay/disputes
Tenant	tenant_id, isolation boundary, policy namespace, data residency/retention settings, key domain, status
Authorization	tenant_id, authorization_id, mandate/action hashes, decision state, approval linkage, policy version, validity/revocation
BudgetReservation	tenant_id, reservation_id, authorization_id, mandate_id, budget bucket, amount/usage reserved, state, pre-execution expiry, commit/release/quarantine reason; expiry may release only when no external execution attempt was submitted
ExecutionAttempt	tenant_id, execution_id, authorization_id, reservation_id, idempotency key, rail reference, status, reconciliation state
ApprovalArtifact	tenant_id, approval_id, step_up_id, principal/approver, agent_id, mandate_id/version/hash, canonical action_hash, challenge nonce, authn method/assurance, issued_at, expires_at, single-use state/signature

6.1 The THAAP Trust Graph

```
PRINCIPAL --delegates--> AGENT --operated_by--> PROVIDER
    |                    |
    |                    +--uses--> TOOL / RUNTIME
    |                    +--presents--> MANDATE
    |                    +--requests--> ECONOMIC ACTION
    |                                       |
    +--owns--> ACCOUNT / WALLET              +--targets--> MERCHANT / PAYEE
                                                |
                                                +--executes_with--> CREDENTIAL
                                                                      |
                                                                      v
                                                                  TRANSACTION
```

Why the graph matters: conventional transaction checks may see individually valid entities. THAAP can identify that the relationship itself is abnormal - for example, a travel agent suddenly purchasing high-value gift cards from a new merchant, outside the semantic intent of the mandate. Neo4j is part of the agreed MVP demonstration because graph context must be available before adaptive risk and step-up. PostgreSQL remains the money/state authority; Neo4j supplies relationship context and graph-derived features, not spend authority.

7. Mandates and Semantic Authorization

The mandate is the machine-verifiable representation of delegated authority. It is the contract between human intent and autonomous execution, but a signed permission is not sufficient by itself: THAAP must have an enforcing execution boundary that can make the mandate operational and reject out-of-scope use.

Constraint family	Examples
Financial	maximum amount, cumulative budget, currency, per-transaction cap
Merchant	allowlist/denylist, merchant ID, category, reputation threshold
Purpose	flight booking, laptop purchase, cloud compute, invoice payment
Product/service	SKU, class of service, quantity, substitution rules
Temporal	not-before, expiry, recurrence cadence, usage count
Geographic	allowed country/region, beneficiary domicile
Data disclosure	what personal attributes the agent may reveal to merchant/provider
Human oversight	approval threshold, biometric/passkey requirement, co-signature
Agent scope	specific agent/provider/key, allowed tools, delegation-to-subagent rules

Example mandate

```
{
  "mandate_id": "mandate_18472",
  "principal_id": "principal_raj",
  "agent_id": "wajo.fo.82714",
  "goal": "buy_one_laptop",
  "constraints": {
    "max_amount": 150000,
    "currency": "INR",
    "allowed_merchants": ["Apple India", "Croma", "Reliance Digital"],
    "quantity": 1,
    "substitution": "DENY",
    "expires_at": "2026-10-06T23:59:00+05:30",
    "step_up_if_amount_gt": 140000
  },
  "proof": "<signed-verifiable-authorization>"
}
```

7.1 Why semantic authorization is different from payment authorization

A transaction may satisfy a numeric budget and still violate user intent. THAAP should compare the proposed action against the meaning of the instruction, not only the amount. Example: a flight under budget that violates "no overnight layover" should be REPLAN or DENY even though the payment is affordable.

> **Rule**
>
> LLMs may interpret intent and generate semantic features; final enforcement must be deterministic and reproducible from stored policy + mandate + evidence.

8. Decision Engine

THAAP should return a richer set of control outcomes than binary payment authorization.

Decision	Meaning	Example
ALLOW	Execute as proposed.	Low-risk purchase fully inside mandate.
ALLOW_WITH_LIMITS	Execute only with a narrower credential or modified limit.	Issue a one-time token for exactly INR 12,500.
STEP_UP	Require human or stronger authentication.	High-value action outside normal autonomous range.
REPLAN	Agent goal is valid but proposed execution violates intent.	Flight has forbidden overnight layover.
REQUEST_EVIDENCE	Need additional proof before deciding.	Merchant identity or agent attestation missing.
DENY	Do not execute.	Gift-card purchase unrelated to laptop mandate.
QUARANTINE	Suspend agent/credential and trigger investigation.	Strong compromise or prompt-injection indicators.

8.1 Decision inputs

- **Agent Trust Score:** identity confidence, attestation, provider reputation, version history, anomaly rate, compromise indicators.
- **Mandate Match:** hard-constraint validation + semantic similarity between user intent and proposed action.
- **Transaction Risk:** merchant/payee risk, amount anomaly, payment credential risk, device/network signals, known fraud patterns.
- **Behavior Deviation:** difference from principal-agent historical behavior and from the agent class baseline.
- **Threat Signals:** prompt injection, suspicious external instructions, tool-chain deviation, credential abuse, unexpected destination.
- **Policy Context:** tenant rules, regulatory constraints, product controls, authentication strength and risk appetite.

8.2 Explainability contract

```
Decision: STEP_UP
Reason codes:
  - AMOUNT_OUTSIDE_NORMAL_AGENT_RANGE
  - NEW_MERCHANT_FOR_AGENT
  - HIGH_VALUE_AUTONOMOUS_ACTION
Required action:
  - PASSKEY_OR_BIOMETRIC_APPROVAL
Evidence:
  - mandate_hash
  - action_hash
  - policy_version
  - agent_attestation_id
  - risk_feature_snapshot
```

9. Agentic Threat Hunting

This is where THAAP can differentiate from pure protocol and token providers. A valid signature does not prove that an agent has not been manipulated. However, a blocked out-of-scope action proves constraint enforcement - not, by itself, prompt-injection detection. THAAP should label prompt injection only when independent telemetry/provenance (for example untrusted-content analysis, instruction lineage, tool-trace change or a dedicated detector) supports that conclusion.

Threat	Detection idea	Control
Prompt injection from merchant/site/email	Independent evidence: untrusted-content/instruction analysis + provenance/taint + causal change in plan/tool trace. A mandate mismatch alone is not sufficient.	Always enforce constraints (deny/replan) when violated. Separately raise PROMPT_INJECTION_SUSPECTED/CONFIRMED only when detector evidence supports it; isolate content and require clean-context re-evaluation.
Credential theft	Valid credential used by wrong agent key/runtime or from abnormal context.	Proof-of-possession; bind credential; revoke.
Mandate replay	Same open authorization reused without valid rejection/closure semantics.	Nonce, idempotency, usage counter, receipt chain.
Authorization drift	Agent gradually broadens interpretation of a standing instruction.	Semantic drift score; cumulative budget/purpose controls.
Tool-chain hijack	Unexpected tool, endpoint or sub-agent appears in execution path.	Tool allowlist + signed provenance + step-up.
Malicious/compromised agent update	Behavior changes after version change or attestation mismatch.	Version trust score, rollout guard, quarantine.
Merchant-agent collusion	Repeated anomalous patterns across merchants/agents/principals.	Graph analytics + ecosystem reputation.
Human-fallback ambiguity	Human operator completes action without clear delegation.	Actor provenance, scoped sub-delegation, auditable receipt.

Critical insight

Cryptographic authorization answers "was authority granted?" Threat hunting answers "is the currently acting system still behaving consistently with that authority?" THAAP needs both.

10. Scoped Execution Credentials

Where integrations allow it, THAAP should issue or broker short-lived capabilities that are useless outside the approved context. A usable credential is created only after durable authorization exists and the relevant budget/usage has been atomically reserved.

```
THAAP-CAPABILITY
tenant          = tenant_acme
agent           = wajo.fo.82714
principal       = principal_raj
authorization   = auth_01J...
budget_resv     = resv_01J...
approval        = approval_01J...  # present when step-up required
action          = purchase
merchant        = Apple India
exact_amount    = INR 146999
purpose         = laptop
valid_for       = 10 minutes
usage           = 1
proof_of_possession = required
replay          = forbidden

Precondition: authorization=COMMITTED AND budget_resv=RESERVED.
```

The underlying payment rail can be a network token, PSP token, UPI flow, wallet authorization or future payment primitive. THAAP should not need to own the settlement instrument, but it must own or integrate with an executor that enforces scope and an execution state machine that reconciles uncertain outcomes before retrying.

10.1 Authorization-to-execution state machine

Permissions become meaningful only when an executor enforces them. THAAP therefore separates a preliminary decision from durable authorization, resource reservation, credential issuance, execution and reconciliation.

```
STATE MACHINE - explicit terminal and non-terminal branches

1. PROPOSED / PRELIMINARY DECISION
   DENY -> DENIED [terminal; no authorization, reservation or credential]
   REPLAN -> NEW_PROPOSAL [new canonical action_hash; evaluate from the start]
   ALLOW -> AUTHORIZATION_COMMITTED
   STEP_UP -> STEP_UP_REQUIRED

2. STEP-UP
   APPROVAL_REJECTED / APPROVAL_CANCELLED / APPROVAL_EXPIRED -> DENIED [terminal]
   APPROVAL_VERIFIED -> RE_EVALUATE
   RE_EVALUATE -> DENIED | REPLAN | AUTHORIZATION_COMMITTED

3. DURABLE AUTHORITY
   AUTHORIZATION_COMMITTED -> BUDGET_RESERVED -> CREDENTIAL_ISSUED
   CANCEL/REVOKE/EXPIRE before any rail submission -> revoke credential + RELEASE reservation

4. EXECUTION
   CREDENTIAL_ISSUED -> EXECUTION_SUBMITTED
   EXECUTION_SUBMITTED -> SUCCEEDED/SETTLED -> COMMIT reservation
   EXECUTION_SUBMITTED -> DEFINITIVE_FAILURE -> RELEASE reservation
   EXECUTION_SUBMITTED -> TIMEOUT/AMBIGUOUS -> PENDING_RECONCILIATION

5. RECONCILIATION
   PENDING_RECONCILIATION -> VERIFIED_SUCCESS -> COMMIT reservation
   PENDING_RECONCILIATION -> VERIFIED_FAILURE / CONFIRMED_CANCELLATION -> RELEASE reservation
   PENDING_RECONCILIATION -> STILL_UNKNOWN -> keep budget QUARANTINED; NO automatic TTL
release

Cancellation after rail submission is not terminal until externally confirmed. Post-
transaction refund/reversal is a separate transition; restoration of mandate budget is an
explicit policy decision.
```

An execution timeout is an unknown state, not a failure. Once an external execution attempt may have reached a payment rail or merchant, THAAP must keep the associated budget quarantined while reconciliation queries the external system using stable idempotency/correlation identifiers. Local TTL or reservation expiry can make a credential unusable for new execution, but it must never by itself release budget from an UNKNOWN/PENDING_RECONCILIATION attempt.

10.2 Approval and transaction binding rules

- Canonical action binding: THAAP canonicalizes the proposed economic action and computes action_hash over the fields that affect authorization - including tenant, principal, agent, mandate/version, action type, merchant/payee/beneficiary, amount, currency, item/SKU or asset, quantity, purpose, recurrence and relevant disclosure constraints.
- Approval binding: a step-up approval signs or otherwise cryptographically binds approval_id and challenge nonce to tenant_id, principal/approver, agent_id, mandate_id/version/hash, decision_id/step_up_id and the exact action_hash. It is single-use and expires independently of the mandate.
- Material-change rule: changing an approval-relevant field after the challenge - for example amount, currency, merchant/payee, beneficiary, item/quantity, purpose or recurrence - invalidates the approval and requires a new authorization/step-up. Tolerances such as tax or shipping must be explicit policy fields, never implicit drift.
- Re-evaluation rule: verified approval is evidence, not final spend authority. THAAP re-checks mandate validity/revocation, current policy, risk, budget and action_hash before committing durable authorization.
- Credential binding: the execution credential binds tenant_id, authorization_id, reservation_id, action_hash, exact execution scope, proof-of-possession key, usage count and expiry. The executor rejects scope/hash mismatch even if the credential signature is valid.
- Transaction/idempotency binding: each external execution attempt has a unique execution_id and a stable rail idempotency key bound to authorization_id + reservation_id + action_hash. A retry must reuse the same logical execution key until reconciliation establishes a terminal outcome; it must not create a second debit.

10.3 Reservation release and cancellation invariants

- Before rail submission: credential expiry, principal cancellation, authorization revocation or reservation expiry may terminate the unused execution path and release the reservation, provided THAAP can prove that no external execution attempt was submitted.
- After rail submission: local expiry never proves non-payment. TIMEOUT/UNKNOWN keeps the reservation quarantined until an authoritative rail/merchant result or explicit operator/dispute resolution establishes SUCCESS, DEFINITIVE_FAILURE or CONFIRMED_CANCELLATION.
- Cancellation after submission is not immediately terminal. It becomes CANCEL_PENDING/PENDING_RECONCILIATION until the external system confirms that no financial effect remains.
- Release is exactly-once and evidence-backed. A reservation is released only on a definitive non-execution/failure/cancellation outcome; it is committed on success. Unknown outcomes remain reserved even after ordinary TTLs expire.
- Post-settlement refund/reversal is handled as a new post-transaction event. Restoring mandate budget or usage is policy-controlled and requires evidence; it is not an automatic consequence of the original reservation lifecycle.

11. Decision Ledger and Replay

Decision Replay should become a flagship capability because disputes in agentic commerce are disputes about delegated intent, not just card possession.

Recorded artifact	Purpose
Principal authentication evidence	Who authorized delegation and at what assurance level?
Agent identity + key + attestation	Which exact agent/runtime acted?
Mandate + signature + version	What was authorized and under what constraints?
Proposed action	What did the agent actually ask to do?
Policy version + evaluation	Which deterministic rules were applied?
Risk features + model versions	What adaptive intelligence influenced the decision?
Decision + reason codes	Why was it allowed/blocked/escalated?
Scoped credential metadata	What capability was granted?
Execution receipts	What ultimately happened on the external rail?
Post-event outcomes	Refund, chargeback, dispute, fraud label, user confirmation.
Durable authorization record	Which decision became enforceable authority, after which approval/re-evaluation, and until when?
Budget/usage reservation	What spending/usage capacity was atomically held before credential issuance, and whether it was committed or released?

Execution/reconciliation state	What rail call was attempted, what idempotency key/reference was used, and how timeout/unknown outcomes were resolved?

```
THAAP DECISION REPLAY
Decision ID: thaap_dec_01H...
Tenant: tenant_acme
Principal: principal_raj
Agent: wajo.fo.82714
Intent: "Buy one MacBook under INR 150,000"
Mandate: VALID
Step-up threshold: amount > INR 140,000
Merchant: Apple India
Amount: INR 146,999
Budget maximum: PASS
Step-up threshold: TRIGGERED
Semantic match: 0.99
Agent trust: 0.97
Transaction risk: 0.04
Preliminary decision: STEP_UP
Approval: PASSKEY VERIFIED at 20:14:32
Re-evaluation: PASS
Durable authorization: COMMITTED
Budget reservation: INR 146,999 RESERVED
Credential: single-use, exact amount/merchant, 10-minute TTL
Execution receipt: VERIFIED
Reservation: COMMITTED/SPENT
Outcome: completed
```

12. Protocol and Ecosystem Strategy

THAAP should integrate with emerging standards without becoming captive to any one standard.

External ecosystem	Use in THAAP
Google AP2	Map open/closed checkout/payment mandates and receipts into THAAP Mandate, Action and Evidence models. AP2 explicitly requires deterministic processing for verification. [4]
Mastercard Verifiable Intent / Agent Pay	Consume or produce proof linking identity, intent and action; enrich with THAAP risk and policy. [5]
Visa Intelligent Commerce	Use agent-bound credentials, authentication and user-instruction controls as execution mechanisms/signals. [6]

Stripe Shared Payment Tokens	Treat seller/amount/time-scoped tokens as downstream execution credentials; add cross-provider policy/risk. [7]
Wajo/Fo	Potential upstream action agent calling THAAP before payment or other sensitive action; ingest Wajo agent identity/mandate context. [1][3]
Future agent protocols	Build adapters to canonical THAAP entities, never fork the core decision engine per protocol.

Interoperability principle

Protocols prove and transport intent. THAAP evaluates whether that intent should be trusted in current context and whether the resulting action should be permitted.

13. Developer Product: Authorization API

The first product must be extremely easy for an agent platform, bank, PSP or merchant to integrate.

13.1 Scenario A - below-threshold purchase (fresh mandate A)

```
POST /v1/authorize

# Scenario A starts from a fresh one-laptop mandate.
{
  "principal": { "id": "principal_raj" },
  "agent": { "id": "wajo.fo.82714", "provider": "wajo" },
  "mandate_id": "mandate_laptop_A",
  "action": {
    "type": "purchase",
    "merchant": "Apple India",
    "amount": 136999,
    "currency": "INR",
    "purpose": "laptop",
    "item": "MacBook Pro",
    "quantity": 1
  },
  "context": {
    "autonomous": true,
    "requested_at": "2026-10-06T21:14:32+05:30"
  }
}
```

13.2 Scenario A - ALLOW and reserve

```
{
  "decision": "ALLOW",
  "decision_id": "thaap_dec_A...",
  "agent_trust_score": 0.97,
  "transaction_risk_score": 0.04,
  "mandate_match": 1.00,
  "authorization": {
    "id": "auth_A...",
    "state": "COMMITTED",
    "action_hash": "sha256:action_A..."
  },
  "budget_reservation": {
    "id": "resv_A...",
    "amount": 136999,
    "state": "RESERVED"
  },
  "capability": {
    "authorization_id": "auth_A...",
    "reservation_id": "resv_A...",
    "action_hash": "sha256:action_A...",
    "scope": "single_transaction",
    "amount": 136999,
    "merchant": "Apple India",
    "usage": 1,
    "expires_in_seconds": 600
  },
  "reason_codes": ["MANDATE_VALID", "BELOW_STEP_UP_THRESHOLD", "LOW_RISK"]
}

# After successful execution, resv_A is COMMITTED/SPENT and the one-laptop usage in
mandate_laptop_A is consumed. Scenario B does not reuse this mandate.
```

13.3 Scenario B - above-threshold purchase (fresh mandate B)

```
POST /v1/authorize

# Scenario B is independent and starts from fresh mandate_laptop_B.
{
  "principal": { "id": "principal_raj" },
  "agent": { "id": "wajo.fo.82714", "provider": "wajo" },
  "mandate_id": "mandate_laptop_B",
  "action": {
    "type": "purchase",
    "merchant": "Apple India",
    "amount": 146999,
    "currency": "INR",
    "purpose": "laptop",
    "item": "MacBook Pro",
    "quantity": 1
  }
}

RESPONSE
{
  "decision": "STEP_UP",
  "decision_id": "thaap_dec_B...",
  "step_up_id": "stepup_B...",
  "action_hash": "sha256:action_B...",
  "reason_codes": [
    "MANDATE_STEP_UP_THRESHOLD_EXCEEDED",
    "HIGH_VALUE_AUTONOMOUS_ACTION"
  ],
  "required_action": {
    "type": "PASSKEY_OR_BIOMETRIC_APPROVAL",
    "bind_to_action_hash": "sha256:action_B...",
    "expires_in_seconds": 300
  },
  "capability": null,
  "budget_reservation": null
}
```

13.4 Scenario B - verified approval, re-evaluation and execution authority

A successful step-up does not directly mint a credential. THAAP persists an approval artifact bound to the exact canonical action hash, re-evaluates the action, commits durable authorization, atomically reserves the applicable budget/usage, and only then issues the scoped credential. Scenario A and Scenario B above are independent fixtures with separate one-laptop mandates; they must not both consume the same single-use mandate or budget reservation.

```
{
  "decision": "ALLOW",
  "decision_id": "thaap_dec_B...",
  "step_up_id": "stepup_B...",
  "approval": {
    "id": "approval_B...",
    "state": "VERIFIED",
    "action_hash": "sha256:action_B...",
    "single_use": true
  },
  "authorization": {
    "id": "auth_B...",
    "state": "COMMITTED",
    "action_hash": "sha256:action_B..."
  },
  "budget_reservation": {
    "id": "resv_B...",
    "amount": 146999,
    "state": "RESERVED"
  },
  "capability": {
    "authorization_id": "auth_B...",
    "reservation_id": "resv_B...",
    "action_hash": "sha256:action_B...",
    "merchant": "Apple India",
    "amount": 146999,
    "usage": 1,
    "expires_in_seconds": 600
  }
}

# If any material action field changes after approval, this approval/capability cannot be
reused; THAAP must re-authorize and, if required, step up again.
```

14. Build Plan: From Prototype to Platform

The first goal is not comprehensive agentic commerce. It is a credible end-to-end authorization loop with verifiable intent, deterministic controls and replay.

Phase	Outcome	What to build
Phase 0 - 1 week	Executable design	Canonical schemas; tenant isolation model; decision taxonomy; threat model; authorization/budget/execution state machines; approval/transaction binding rules; OpenAPI; Neo4j graph schema/queries; fake agent + merchant/rail simulator.

Phase 1 - 3 to 4 weeks	MVP Authorization Gateway	Tenant-scoped gateway, agent/principal registry, mandate service, Neo4j Trust Graph available before risk/step-up, deterministic policy, step-up, durable authorization + budget reservation, scoped credential, enforcing mock executor, reconciliation worker, decision ledger/replay.
Phase 2 - 3 to 4 weeks	Adaptive trust	Adaptive risk, semantic mandate match, graph-informed anomaly features, independent prompt-injection telemetry/provenance + detector evaluation, quarantine workflow.
Phase 3 - 4 to 6 weeks	Protocol interoperability	AP2 adapter, mock network/PSP adapter, scoped capability broker, signed receipts.
Phase 4	Graph hardening + SOC	Productionize Neo4j topology/backup/tenant controls and graph projection pipeline; richer cross-entity features, agent-version reputation, threat hunting and quarantine workflows.
Phase 5	Commercial hardening	Enterprise tenancy hardening (not first introduction), SLA/HA, compliance controls, audit export, SDKs, partner integrations, pricing/metering and optional stronger tenant key isolation.

14.1 Independent MVP demo scenarios

1. Scenario A starts from fresh mandate_laptop_A: one laptop under INR 150,000 from an allowlisted merchant set, with step-up required above INR 140,000.
2. Scenario A: the agent proposes INR 136,999. THAAP validates the mandate, commits durable authorization, reserves budget, issues an enforceable scoped capability and, on verified success, commits the reservation and consumes the one-laptop usage.
3. Scenario C is a separate clean fixture: an agent exposed to a malicious page proposes unrelated gift cards. THAAP blocks the action because it violates purpose/merchant constraints. This proves

constraint enforcement; prompt-injection attribution is claimed only when separate telemetry/provenance supports it.

4. Scenario B starts from independent fresh mandate_laptop_B: the agent proposes INR 146,999, triggering deterministic passkey step-up. The verified approval is bound to the exact action hash; THAAP re-evaluates, commits authorization, reserves budget, issues the credential and executes. It does not reuse mandate_laptop_A.
5. Replay reconstructs tenant, mandate, graph/risk context, preliminary decision, step-up approval, durable authorization, budget reservation, credential, execution receipt and any reconciliation events.

Demo success criterion

A viewer should understand in under five minutes that THAAP is not another fraud score. It is the decision boundary that safely converts delegated AI intent into permission to move money.

15. Suggested Technical Architecture

The implementation should optimize for auditability, low latency, deterministic replay and the ability to evolve ML independently from policy enforcement.

Layer	Suggested direction
API / services	Java + Spring Boot or Kotlin for core authorization services; gRPC internally where useful; REST/OpenAPI externally.
Policy	OPA/Rego or a constrained internal DSL for deterministic policy; version every policy bundle used in a decision.
Eventing	Kafka for decision/outcome events, threat signals, model features and durable PostgreSQL -> Neo4j graph projection events. Use an outbox/CDC pattern so graph updates are not silently lost.
Operational data	PostgreSQL/Aurora PostgreSQL is authoritative for tenant-scoped principals, agents, mandates, approvals, authorizations, budget reservations, policies and execution/reconciliation state. Enforce tenant_id in schema/access paths and RLS/authorization where appropriate.
Fast state/cache	Redis for tenant-scoped nonce/replay protection, short-lived credential state, rate limits and hot risk features; never use cache-only state as durable authority or budget truth.

Evidence ledger	Tenant-partitioned append-only evidence records in immutable/object storage with hashes/signatures; relational index for lookup; include approvals, reservations, execution and reconciliation.
Graph	Neo4j from the first demonstration/MVP for tenant-scoped principal-agent-provider-counterparty-credential-action relationships and graph-derived risk features. PostgreSQL remains transactional authority; Neo4j is a risk-context projection, updated via transactional outbox/Kafka or equivalent durable propagation.
ML / semantic risk	Python services for feature pipelines and model inference; keep outputs as signals, not direct money-moving authority.
Cryptography	KMS/HSM-backed signing keys; JWS/JWT/SD-JWT-compatible primitives; key rotation and revocation.
Observability	OpenTelemetry traces with decision_id correlation; Prometheus metrics; immutable audit logs; security alerts.
Deployment	Kubernetes for control plane services; tenant isolation middleware/policy from first implementation; isolate key/credential operations; multi-AZ first, multi-region when product/SLA demands it.
Tenancy	tenant_id is a mandatory key on domain objects/events; tenant-scoped authz, policy namespaces, database filters/RLS, cache prefixes, object-store partitions, graph edges and audit/replay access from day one.
Execution / reconciliation	Explicit state machine with authorization_id, reservation_id, execution_id and rail idempotency key. Timeout -> PENDING_RECONCILIATION, never immediate blind retry; poll/query rail until terminal, then commit/release reservation.

15.1 Non-functional targets for the prototype

- **Determinism:** Given the same stored inputs, policy version and model outputs, replay yields the same control decision.
- **Idempotency: Retries never create multiple authorizations, reservations, credentials or executions. External rail calls carry stable idempotency/correlation keys.**

- **Fail closed for hard controls:** Unverifiable mandate, unknown constraint or invalid signature cannot silently degrade to allow.
- **Low-latency core:** Keep cryptographic and deterministic evaluation on the synchronous path; expensive investigation runs asynchronously unless needed for risk.
- **End-to-end traceability: Every request, graph/risk evaluation, step-up, durable authorization, budget reservation, credential, execution attempt, receipt and reconciliation shares a decision_id/correlation chain.**
- **Privacy and tenant isolation: Selective disclosure and data minimization by default; every data path is tenant-scoped from the first implementation, not postponed to commercial hardening.**
- Durable authority: A credential may be usable only when its linked authorization is committed/not revoked and its budget/usage reservation is active; revocation or reservation release invalidates future use.
- Execution correctness: A signed permission is inert until an enforcing executor applies it. A timeout is UNKNOWN/PENDING_RECONCILIATION, not a failed payment; no blind retries.

16. Competitive Positioning and Moat

THAAP should avoid competing where incumbents already have structural advantage and instead become the neutral layer that improves every participant.

Player type	They naturally own	THAAP should own
Agent platforms	User experience, planning, memory, tools, action execution.	Independent authorization, risk and proof before economic execution.
Card networks	Network identity, tokenization, authorization rails, network fraud data.	Cross-network + cross-agent trust intelligence and policy orchestration.
PSPs	Merchant checkout, tokens, payment orchestration, merchant risk.	Principal-agent mandate semantics, cross-provider agent reputation, decision replay.
Banks / issuers	Account/customer identity, balances, issuer authorization and fraud.	Reusable agent risk/intent layer across banks, merchants and protocols.
Identity vendors	Authentication and credentials.	Economic delegation + behavioral trust + transaction policy + evidence chain.

16.1 Potential moat flywheel

```
More agent platforms / banks / merchants
                |
                v
More authorized + blocked + disputed agent actions
                |
                v
Richer Trust Graph + better threat intelligence
                |
                v
Better decisions + lower friction + stronger evidence
                |
                v
More ecosystem adoption
```

The defensible asset is not a single model. It is the combination of canonical trust primitives, cross-ecosystem graph data, behavioral histories, policy integrations, evidence infrastructure and operational credibility in high-stakes money movement.

17. Initial Buyers and Business Model

Buyer	Pain	Possible commercial model
Agent platforms	Need safe payments/actions without building a financial trust stack.	Per authorization + premium risk/evidence tiers.
Banks / issuers	Need to distinguish legitimate agent transactions from fraud and enforce delegation.	Enterprise platform + per-decision usage.
PSPs / acquirers	Need agent-aware risk and dispute evidence across merchants.	API usage + risk modules.
Large merchants / marketplaces	Need to accept high-quality agents while blocking bots/compromised agents.	Per authorization / per protected GMV.
Enterprises	Need controls for procurement, AP, treasury and machine spending.	Per agent / policy seat + transaction usage.

18. Product and Risk Metrics

Metric	Why it matters
Authorization latency P50/P95/P99	THAAP cannot become the bottleneck for autonomous execution.
False allow rate	Unsafe actions incorrectly permitted.
False step-up / false deny rate	Measures friction imposed on valid autonomous activity.
Mandate mismatch detection rate	Core semantic authorization quality.

Confirmed prompt-injection detection quality	Measure only against labeled/provenance-backed injection events; do not infer detection success from a blocked constraint violation.
Credential misuse / replay prevented	Value of scoped capabilities and proof-of-possession.
Dispute evidence completeness	Percentage of disputed actions reconstructable end-to-end.
Agent trust calibration	Whether trust scores predict future unsafe outcomes.
Time to revoke / quarantine	Operational containment effectiveness.
Autonomous completion uplift after safe step-up/replan	Shows THAAP improves outcomes, not only blocks risk.
Constraint enforcement rate	Percentage of out-of-scope amount/merchant/purpose/time/action attempts correctly blocked or replanned regardless of root cause.
Budget reservation correctness	No overspend/double-reservation under retries/concurrency; reservations are committed/released exactly once.
Unknown-payment reconciliation time	Time from timeout/unknown execution result to a verified terminal rail outcome; detects unsafe retry behavior.
Tenant isolation violations	Must remain zero; continuously test authorization, storage, cache, graph and evidence paths for cross-tenant leakage.
Unknown-budget auto-release violations	Must remain zero. No reservation tied to an ambiguous submitted execution may be released solely because a local TTL or reservation expiry elapsed.

19. What THAAP Should Not Build First

- A general consumer shopping assistant.
- A ChatGPT/Claude/Gemini competitor.
- A browser automation framework as the core product.
- A new card network or settlement rail.
- A full merchant checkout stack.
- Graph complexity without a demonstrated trust/risk use case. Use Neo4j for the agreed Trust Graph demonstration, but avoid premature clustering, graph ML infrastructure or cross-tenant global graph features until the use cases and operating model justify them.
- A black-box LLM that directly returns ALLOW/DENY with no deterministic policy layer.
- A huge fraud model before the mandate/identity/evidence foundation works end-to-end.

Focus rule

If a feature makes THAAP better at proving authority, constraining action, detecting misuse, deciding risk or explaining an outcome, it is probably core. If it mainly makes an agent more capable at shopping, browsing or chatting, it is probably not.

20. Key Risks and Mitigations

Risk	Mitigation
Standards evolve quickly	Canonical internal model + adapter architecture; do not hard-code AP2/UCP semantics into core domain.
Networks/PSPs build overlapping features	Be cross-network, cross-agent and risk/policy focused; integrate rather than replace.
Insufficient early data for ML	Start with deterministic mandate/policy checks + simulation; introduce learning after evidence accumulates.
Privacy / sensitive data concentration	Selective disclosure, tokenization, hashes, retention minimization and tenant isolation from day one across storage/cache/graph/evidence; stronger cryptographic isolation as product matures.
LLM non-determinism	Never make model output the only authority for money movement; persist model version/output as an advisory signal.
Latency from too many checks	Tiered decision path; cache stable identity/reputation; move non-critical enrichment async.
Ambiguous liability	Evidence-first design, explicit role/delegation model, clear integration contracts and immutable receipts.
Low adoption by agent developers	One-call API, SDKs, sandbox, protocol adapters and a compelling replay/debugging experience.
Payment timeout / ambiguous rail outcome	Treat timeout as UNKNOWN; use stable idempotency keys and reconciliation queries before retry. Once an execution may have been submitted, keep budget quarantined until a verified terminal rail outcome or explicit operator/dispute resolution; local TTL/expiry must never auto-release unknown-payment budget.

Permission without enforcement	Never treat a signed mandate or ALLOW response as self-enforcing; credential/executor must check authorization state, reservation and scope at use time.

21. Recommended 90-Day Founder/CTO Execution Plan

Window	Founder/CTO objective	Deliverable
Days 1-15	Nail the primitive	Canonical model, tenant isolation contract, threat model, causal sequencing ADR, decision taxonomy, mandate schema, complete authorization/budget/execution state machines, approval/transaction binding invariants, OpenAPI, Neo4j schema and demo queries.
Days 16-45	Ship the enforceable trust boundary	Tenant-scoped /authorize path, Neo4j Trust Graph/context before risk/step-up, deterministic policy, step-up, durable authorization, atomic budget reservation, scoped credential, enforcing executor/rail simulator, strict unknown-payment reconciliation, decision ledger/replay.
Days 46-70	Make risk graph-informed and agent-native	Semantic intent match, relationship/anomaly signals before step-up, independent prompt-injection telemetry/provenance and detector evaluation, quarantine/replan reason codes.
Days 71-90	Prove interoperability	AP2-compatible demo adapter, agent SDK, merchant/PSP simulator, polished technical demo and design-partner pitch.

21.1 The demo I would show investors/design partners

1. Scenario A uses a fresh one-laptop mandate with a maximum of INR 150,000 and mandatory step-up above INR 140,000; Neo4j already contains the tenant-scoped principal-agent-merchant trust context used before risk/step-up.
2. Scenario A proposes INR 136,999: THAAP evaluates Neo4j graph context + policy/risk, commits durable authorization, reserves budget, issues an enforceable scoped capability and commits the reservation after verified execution.
3. A malicious page causes the agent to propose unrelated gift cards: THAAP blocks the constraint violation. Separately, prompt-injection telemetry shows whether THAAP can attribute the deviation to malicious instructions; the demo does not conflate the two.
4. Scenario B starts from a separate fresh one-laptop mandate and proposes INR 146,999: the threshold deterministically triggers passkey step-up. The approval binds to the exact action hash; after re-evaluation THAAP commits authorization, reserves budget, issues the credential and executes.
5. The operator opens Decision Replay and sees tenant, intent, graph context, decision, approval, reservation, credential, external execution and reconciliation/receipt chain.
6. The same demo is repeated through an AP2 adapter to prove protocol interoperability.

22. Final Product Vision

THAAP wins if it becomes infrastructure that users never need to think about but every serious autonomous transaction can rely on - not only to express permission, but to enforce it, reserve scarce authority/budget, execute safely, reconcile uncertainty and prove the outcome. Unknown external payment state is treated as conserved uncertainty: authority may expire, but potentially spent budget is not released without evidence.

> **THAAP mission**
>
> Make autonomous economic action trustworthy enough to become normal.

The long-term category: Trust infrastructure for the autonomous economy.

The long-term boundary THAAP should own: the moment where probabilistic machine intent becomes permission to create a real-world financial consequence.

The five guarantees: Attributable. Authorized. Constrained. Explainable. Recoverable.

References

These sources validate the market direction and external protocol landscape. They are references, not implementation dependencies.

[1] Wajo - Fo product page - https://wajo.ai/

[2] Wajo - Unveiling Wajo - https://wajo.ai/blog/unveiling-wajo/

[3] Wajo Terms of Service - agent actions, delegation and payments - https://wajo.ai/terms-of-service

[4] Google Agent Payments Protocol (AP2) specification - https://github.com/google-agentic-commerce/AP2/blob/main/docs/ap2/specification.md

[5] Mastercard - Verifiable Intent - https://www.mastercard.com/us/en/news-and-trends/stories/2026/verifiable-intent.html

[6] Visa Intelligent Commerce - https://www.visa.com/en-us/solutions/intelligent-commerce

[7] Stripe - Agentic Commerce Suite / Shared Payment Tokens - https://stripe.com/blog/agentic-commerce-suite

[8] TechCrunch - Vinod Khosla on Wajo and trust - https://techcrunch.com/2026/10/06/vinod-khosla-believes-ex-deepmind-engineers-wajo-will-win-agent-market-on-trust/

Prepared as a THAAP product strategy and technical architecture working document. External product specifications and standards will evolve; protocol integrations should be versioned and validated against current specifications before production use.